PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 1
2. Background & Business Objective 1
3. Product Purpose 2
4. System Functionalities 2
5. User Stories & Use Cases 3
6. Features Included (Scope Definition) 3
7. Features Not Included (Scope Control) 3
8. Assumptions & Constraints 4
9. Risks & Questions Throughout Development 4

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: AI Rent Advisor

Version: 2.0

1. Project Overview

- **Problem Statement:** In Malaysia, many renters make housing decisions based on incomplete property listings and scattered online information such as forums, reviews, and social media comments. This fragmented data makes it difficult to evaluate the true value of a rental property, leading to poor decisions, hidden costs, and exposure to risky landlords or unsafe living conditions.
- **Target Domain:** Artificial Intelligence for Economic Empowerment and Decision Intelligence, specifically in the rental housing domain.
- **Proposed Solution Summary:** AI Rent Advisor is an AI-powered rental decision intelligence system that integrates structured data (such as rent, location, and property features) with unstructured data (such as tenant reviews and complaints). By leveraging the Z.ai GLM model, the system analyzes multiple sources of information to generate a clear rental verdict such as "Great Deal", "Acceptable", or "Avoid". In addition, it provides hidden cost estimation, risk detection, and negotiation advice, enabling users to make informed and data-driven rental decisions.

2. Background & Business Objective

- **Background of the Problem:** The current rental decision-making process is inefficient and fragmented. Most rental platforms only provide basic information such as price, images, and location, while critical insights such as tenant experiences, hidden costs, and risks are scattered across different sources. Users must manually search, interpret, and compare this information, which is time-consuming and often unreliable. Furthermore, many renters lack the knowledge or tools to evaluate whether a property is fairly priced or potentially risky.

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Importance of Solving This Issue:** Solving this problem can significantly improve the quality and efficiency of rental decision-making. By automating data analysis and consolidating information into a single platform, users can quickly identify overpriced properties, hidden costs, and potential risks. This reduces financial losses, saves time, and empowers users to make better decisions without requiring expert knowledge.
- **Strategic Fit / Impact:** This project aligns with the concept of AI-driven decision intelligence systems, where artificial intelligence supports users in making complex real-world decisions. By integrating Z.ai GLM, the system demonstrates how AI can transform both structured and unstructured data into actionable insights, ultimately improving economic outcomes such as cost savings, better budgeting, and smarter resource allocation.

3. Product Purpose

3.1. Main Goal of the System: The main goal of the system is to provide users with an intelligent assistant that evaluates rental properties, identifies hidden costs and risks, and generates clear, explainable recommendations to support better decision-making.

3.2. Intended Users (Target Audience):

- University students
- Young professionals
- First-time renters
- Budget-conscious individuals
- Individuals with limited experience in evaluating rental properties

4. System Functionalities

4.1. Description: The system is an AI-powered rental intelligence platform designed to help tenants make safer and more informed renting decisions in Malaysia. It operates by allowing

© UMHackathon 2026

users to input a property listing URL or natural language queries. The system then extracts relevant listing data, combines it with external unstructured sources such as tenant forums, social media discussions, and market benchmarks, and generates clear, human-readable insights.

Using a GLM-based AI agent, the system interprets rental information, detects hidden costs, evaluates landlord reliability patterns, and produces structured decision outputs. The goal is to simplify complex rental information into an easy-to-understand verdict, ensuring users can quickly understand the true cost, risks, and fairness of a property before committing.

4.2. Key Functionalities:

- **Rental Verdict (Decision Engine):**
  Provides a final recommendation — *Great Deal, Acceptable, or Avoid* — based on rental price comparison, affordability analysis, and aggregated tenant feedback.

- **True All-In Cost Calculator:**
  Calculates the actual monthly rental cost by including hidden expenses such as parking fees, deposits, maintenance charges, utility setup costs, and access card fees.

- **Risk Radar:**
  Analyzes unstructured data from forums, Facebook groups, and reviews to detect recurring landlord or property issues such as deposit disputes, maintenance failures, or management problems.

- **Negotiation Coach:**
  Generates personalized negotiation strategies, including recommended target rent, key bargaining points, and ready-to-use message templates for communicating with landlords.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: The system uses **Z.ai GLM (General Language Model)** as the core AI engine. GLM was selected due to its strong natural language understanding,

summarization capability, and ability to perform structured reasoning over mixed data sources (listing text, reviews, and user queries). These capabilities make it suitable for interpreting unstructured rental information and generating consistent decision outputs such as verdicts, cost breakdowns, and risk evaluations.

4.3.2. **Prompting Strategy:** The system adopts a **multi-step agentic prompting strategy**. Instead of relying on a single prompt, the AI agent processes the input in stages:

1. Extract listing and contextual data
2. Analyze pricing, risks, and hidden costs separately
3. Synthesize results into final outputs (verdict, cost, risk, and negotiation advice)

This approach ensures better accuracy, reduces hallucination, and allows modular reasoning aligned with each feature of the system.

4.3.3. **Context & Input Handling**: The system accepts two main inputs: property listings, and monthly income. For large or unstructured inputs (e.g., long listings or multiple reviews), the system applies chunking and selective extraction to retain only relevant rental information.

A maximum input threshold is defined for processing; inputs exceeding this limit are truncated and prioritized based on relevance (price, location, and user query context). Irrelevant or overly large inputs are filtered out before being passed to the AI model to ensure stable performance and response quality.

4.3.4. **Fallback & Failure Behavior:** To maintain reliability, the system includes fallback mechanisms when the AI model produces off-topic, incomplete, or hallucinated responses. In such cases, the system will:

- Trigger a retry with refined prompts
- Re-validate outputs against structured rules (e.g., cost consistency checks)

© UMHackathon 2026

- Return a simplified fallback response if confidence is low (e.g., "Insufficient data to generate a reliable verdict")

If repeated failures occur, the system gracefully degrades output quality rather than producing misleading information, ensuring user safety and trustworthiness of recommendations.

5. User Stories & Use Cases

- **User Stories:**

Role	*User Stories*
End Users (Renters / Tenants)	• *As a renter, I want to input a property listing and my income so that I can know whether the rental is affordable for me.* • *As a renter, I want to see a clear verdict (Great Deal / Acceptable / Avoid) so that I can make quick rental decisions without analysing data myself.* • *As a renter, I want to see the true monthly cost including hidden fees so that I understand the real financial burden before signing a contract.* • *As a renter, I want to check risk warnings from past tenant complaints so that I can avoid problematic landlords or properties.* • *As a renter, I want negotiation suggestions so that I can reduce rental prices before committing to a unit.* • *As a renter, I want to compare up to 3 properties so that I can choose the best option side-by-side.*
Landlords	• *As a landlord, I want to understand how my property compares to market rent so that I can price it competitively.* • *As a landlord, I want to identify "good tenant vs risky tenant"* • *profiles so that I can reduce vacancy and protect my property.* • *As a landlord, I want to set flexible deposit requirements based on tenant risk so that I can balance occupancy speed and property safety.* • *As a landlord, I want to see tenant negotiation feedback so that I can adjust pricing strategies for faster rental.*
System / AI Behaviour	• *As the system, I want to classify rental outcomes into structured labels (Great Deal / Acceptable / Avoid) so that frontend can display consistent decisions.* • *As the system, I want to convert raw rent and dataset signals into*

© UMHackathon 2026

	human explanations so that users understand the reasoning clearly. ● *As the system, I want to generate negotiation talking points so that tenants can communicate effectively with landlords.* ● *As the system, I want to process complaint signals into risk scores so that risky properties are flagged early.*

- **Use Cases (Main Interactions):**

Use Case 1: Generate rental verdict
Use Case ID: UC-01
Actor: Tenant (User)

Precondition:

1. User has access to the application
2. Backend server and AI module are available

Postcondition:

1. System generates a rental verdict (Great Deal / Acceptable / Avoid)
2. Explanation is displayed on the dashboard

Main Flow:

1. User enters property details (location, rent, income).
2. Frontend sends request to /analyse endpoint.
3. Backend retrieves market benchmark from dataset.
4. Financial & Risk Engine calculates affordability and cost signals.
5. AI Engine (GLM) processes inputs and generates verdict + explanation.
6. Backend combines all results into structured JSON.
7. Frontend displays:
   - Verdict card
   - Explanation

Alternative Flow:

A1: User inputs incomplete data

- System prompts user to complete missing fields
- Flow returns to Step 1

© UMHackathon 2026

A2: User selects comparison mode instead

- System redirects to /compare endpoint
- Use Case UC-02 is triggered

Exception Flow:

E1: GLM API timeout (>15 seconds)

- System retrieves cached response from cache_service
- Displays fallback result

E2: Backend failure / server error

- System displays error message: "Service temporarily unavailable"

Use Case 2: Calculate true all-in cost calculator
Use Case ID: UC-02
Actor: Tenant (User)

Precondition:

1. User has entered rental price and property details
2. Financial engine is operational

Postcondition:

1. System displays total monthly cost including hidden fees

Main Flow:

1. User submits property listing price.
2. Backend calls financial_risk_engine.js.
3. System calculates:
   - Parking fees
   - Utilities
   - Maintenance
   - Deposits (converted monthly)
4. System aggregates total monthly cost.
5. Result is returned to frontend.
6. Frontend displays cost breakdown card.

Alternative Flow:

© UMHackathon 2026

A1: Missing optional cost data

- System uses estimated default values
- Continues calculation

Exception Flow:

E1: Calculation service failure

- System returns partial cost with warning
- Displays: “Estimated values used”

Use Case 3: Detect risk
Use Case ID: UC-03
Actor: Tenant (User) / Landlord

Precondition:

- Property data is available
- Complaint from online resource is loaded in the system

Postcondition:

- System generates a risk score and warning message

Main Flow:

1. User submits property details.
2. Backend retrieves related complaint data.
3. Risk engine scans for repeated complaint patterns.
4. System assigns severity score (1–10).
5. AI Engine converts data into human-readable warning.
6. Frontend displays risk radar and explanation.

Alternative Flow:

A1: No complaint data found

- System marks property as “Low Risk (Insufficient Data)”
- Continues to display result

Exception Flow:

© UMHackathon 2026

E1: Dataset unavailable

- System skips risk analysis
- Displays message: “Risk data temporarily unavailable”

Use Case 4: Negotiate rental
Use Case ID: UC-04
Actor: Tenant (User)

Precondition:

- Rental verdict and risk analysis have been generated

Postcondition:

- System provides negotiation suggestions and talking points

Main Flow:

1. System receives verdict, cost, and risk outputs.
2. AI Engine generates negotiation strategies.
3. System determines:
   - Suggested target price
   - Key negotiation reasons
4. AI generates sample messages to landlord.
5. Frontend displays a negotiation card.

Alternative Flow:

A1: Property is “Great Deal”

- System suggests minimal or no negotiation
- Displays message: “Price already competitive”

Exception Flow:

E1: AI generation fails

- System displays fallback negotiation tips template

Use Case 5: Suggest risk-based security deposit structure
Use Case ID: UC-05
Actor: Landlord

© UMHackathon 2026

Precondition:

- Tenant financial and profile data is available
- Affordability analysis has been completed
- System AI module is operational

Postcondition:

- System classifies tenant as Good or Risky
- Deposit recommendation and rental strategy are displayed

Main Flow:

1. Landlord inputs or views tenant profile data.
2. System evaluates tenant affordability and financial stability.
3. Financial & Risk Engine processes tenant indicators.
4. AI Engine classifies tenant into:
   - Good Tenant
   - Risky Tenant
5. System assigns deposit recommendation:
   - Good Tenant → 1–2 months deposit
   - Risky Tenant → 3–4 months deposit
6. System generates rental strategy recommendations.
7. Frontend displays classification and suggestions to landlord.

Alternative Flow:

A1: Insufficient tenant data

- System uses estimated/default indicators
- Displays "Assessment based on limited data"

Exception Flow:

E1: AI classification fails

- System falls back to rule-based classification
- Displays generic recommendation

Key System Interaction Flow:

Prompt → AI Processing → Structured Output → UI Rendering

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. User enters property details
2. Backend collects dataset + risk + cost signals
3. Member 1 AI engine processes reasoning via GLM
4. Outputs structured JSON:
   - verdict
   - cost breakdown
   - risk score
   - negotiation advice
5. Frontend renders:
   - verdict card
   - cost breakdown card
   - risk radar
   - negotiation coach

Flow Generation Behaviour:

- Input is unstructured user text + numeric property data
- System:
  1. Parses intent (analyse / compare / cost / risk)
  2. Routes to correct backend module
  3. Maintains consistent "AI token style" output:
     - verdict labels
     - risk scores
     - explanation text
  4. Ensures same formatting across all screens

6. Features Included (Scope Definition)

- **Automated Listing Extraction:** A backend service that scrapes or parses structured data from pasted PropertyGuru/iProperty URLs.

© UMHackathon 2026

- **GLM-Powered Verdict Engine:** The central "brain" using **Z.ai GLM** to weigh market benchmarks against tenant complaints to produce a labeled verdict (GREAT DEAL / ACCEPTABLE / AVOID).
- **Unstructured Risk Analysis:** A text-processing module that scans Facebook groups and forums to detect systemic patterns of landlord neglect or withheld deposits.
- **True All-In Cost Calculator:** A financial logic layer that calculates the "Real Number" by adding hidden fees like unbundled parking and out-of-pocket maintenance.
- **Negotiation Strategy Generator:** A tool that creates a human-language script for renters based on the AI's findings.
- **Bilingual Processing (EN/BM):** The system must understand and process both English and Bahasa Malaysia inputs and sources.
- **Cross-Platform UI (Flutter):** A responsive mobile/web dashboard to visualize all AI insights, costs, and risks.

7. Features Not Included (Scope Control)

- **Real-Time Data Scraping:** The system will use a pre-processed Kaggle dataset for market benchmarks instead of live-crawling the entire web in real-time.
- **Legal & Contract Review:** The AI will not provide legal advice or review actual tenancy agreements.
- **Property Booking/Payments:** Users cannot pay deposits or book viewings through the app.
- **Account History Sync:** While comparisons can be saved locally, a full cloud-synced user profile system with photo uploads is excluded.
- **In-App Messaging:** There will be no live chat function between the renter and the landlord or agent.
- **Physical Commute Tracking:** The "Commute Reality Check" is based on static map and news data, not real-time GPS tracking of MRT trains.

8. Assumptions & Constraints

© UMHackathon 2026

- **LLM Cost Constraint:**
  - **Estimated token usage per session:**
    ~2,000–3,500 tokens per user session (includes verdict, cost explanation, risk narrative, and negotiation prompts).
  - **Estimated cost strategy:**
    Since each analysis triggers multiple prompts, costs scale linearly with usage.
  - **Cost-control design decisions:**
    - **Centralised AI access**: prevents redundant or duplicated API calls.
    - **Response caching**: repeated queries (same property inputs) return cached GLM outputs instead of re-calling the model.
    - **Prompt minimisation**: prompts are structured and concise (no unnecessary context or verbose instructions).
    - **Batching outputs**: multiple outputs (verdict + explanation + negotiation tips) are generated in fewer API calls instead of separate requests.

- **Technical Constraints:**
  - The system relies on external GLM API availability (Z.ai GLM), meaning:
    - Downtime or rate limits will affect system functionality.
  - Frontend is built using Flutter Web, which:
    - May have limited compatibility with some browser features compared to native apps.
  - Backend uses Node.js / Python APIs, with:
    - Limited real-time processing (no streaming responses implemented).
  - Authentication mechanism is simplified and not production-grade:
    - Auth system may be basic (token/local storage) and not production-grade.
  - Privacy Constraint: Since you are handling "Monthly Income" and "Asking Rent," you should note that PII (Personally Identifiable Information) is not stored (or is anonymized) before being sent to the Z.ai GLM to ensure user privacy.

© UMHackathon 2026

- Connectivity: Flutter Web apps can be heavy. A constraint to add is that the application requires a stable high-speed internet connection to fetch the GLM responses.

- **Performance Constraints:**
  - AI latency:
    Target average latency: <5s, with fallback triggered at 8s.
  - Fallback mechanism:
    Cached responses are used if API response exceeds ~8 seconds.
  - Scalability limitation:
    High concurrent users may overload:
    - API rate limits
    - Backend processing
  - Frontend limitation:
    Complex UI components (charts, radar visuals) may slow rendering on low-end devices.

- **User Input:**
  - The system depends heavily on accurate user input, including:
    - Property name
    - Location/postcode
    - Asking rent
    - Monthly income
  - Incorrect or unrealistic inputs will lead to:
    - Misleading affordability analysis
    - Inaccurate verdicts
  - The AI:
    - Does not verify real property listings
    - Generates advisory insights, not guaranteed financial decisions

© UMHackathon 2026

- Human judgment is still required:
  - The tool is decision-support, not a final authority

9. Risks & Questions Throughout Development

- **Design Similarity:**
  - **Risk:**
    - AI-generated outputs (verdicts, explanations, negotiation advice) may appear too similar across different users or property inputs, reducing perceived personalization and credibility.
  - **Questions:**
    - How do we ensure the AI produces varied and context-specific explanations?
    - Will repeated prompts result in identical or generic responses?
  - **Mitigation:**
    - Use structured inputs (price, income, risk score) to drive variation
    - Design multiple prompt templates in the AI module
    - Combine rule-based scoring (from Financial & Risk Engine) with AI-generated explanations
    - Slightly adjust prompt phrasing while maintaining consistency in verdict labels

- **Frame Stability:**
  - **Risk:**
    - The system may break or behave inconsistently due to integration issues, slow API responses, or mismatched data between frontend and backend.
  - **Questions:**
    - What happens if the GLM API is slow or fails during analysis?
    - Will the UI remain stable if data is incomplete or delayed?

© UMHackathon 2026

        - Can all modules (frontend, backend, AI, financial engine) communicate reliably?
    - **Mitigation:**
        - Implement cached fallback responses for demo scenarios
        - Use loading skeleton screens instead of blank states
        - Define a strict and consistent JSON response structure
        - Add error handling and default values in backend
        - Perform daily integration testing across all modules

- **Drawing Interpretation**:
    - **Risk:**
        - The system may misinterpret user inputs (e.g., unrealistic income, incorrect rent values) or oversimplify risk detection due to reliance on mock datasets and keyword matching.
    - **Questions:**
        - What happens if user input is incomplete, unrealistic, or inaccurate?
        - Can the system correctly interpret affordability and risk in all cases?
        - How reliable is the risk analysis based on limited or simulated data?
    - **Mitigation:**
        - Implement input validation on both frontend and backend
        - Use conservative financial assumptions for cost calculations
        - Clearly explain how results are derived (transparency via AI explanations)
        - Label outputs as “estimates” or “guidance” rather than absolute conclusions
        - Design system for future improvement with real datasets

- **AI Hallucination Risk**
    - **Risk:**

© UMHackathon 2026

- AI may generate explanations that sound convincing but are not fully aligned with actual calculations.

- **Mitigation:**
  - Anchor AI outputs to structured backend data
  - Pass computed values explicitly into prompts
  - Avoid letting AI "infer" financial numbers

© UMHackathon 2026

Email: umhackathon@um.edu.my